



INDIA.RUNS

Build what next India runs on

Team Name : LEO

Team Leader Name : KOTA SRIRAMA MANISH

Problem Statement : Recruiters screen thousands of profiles yet miss the right candidates – keyword filters cannot understand what a role truly needs. RedroBRanker solves this with neural retrieval.

Solution Overview

- RedroBRanker is a three-stage neural IR pipeline - Title Pre-Filter, Bi-Encoder Semantic Retrieval, and Cross-Encoder Re-Ranking - that ranks 100,000 candidate profiles in under 90 seconds on CPU, finding 745 genuine ML/AI engineers hidden within a 100K dataset where 69% of profiles are completely irrelevant non-tech roles.
- Traditional systems match keywords. RedroBRanker understands meaning - 60% of the skill score is semantic similarity, not keyword presence. A candidate who built a recommendation engine ranks alongside one who lists FAISS and dense retrieval. Local cross-encoder re-ranking provides deep relevance judgment with zero external API dependencies during ranking.

JD Understanding & Candidate Evaluation

Six Weighted skill groups for evaluating the candidate

Skill Group	Weight
Embeddings & Dense Retrieval	20%
Vector DB & ANN Search	20%
Retrieval & Ranking Systems	20%
LLMs & NLP	15%
Evaluation (NDCG / MAP)	15%
Python & Production Eng.	10%

- Six disqualifier rules converted to soft penalty curves - all-consulting career, pure research with no production, recent LangChain-only AI, non-coding leadership, CV/Speech-only background, closed-source-only 5+ years
- Three targeted JD embedding queries capture technical fit (60%), production mindset (30%), and domain context (10%) * finding plain-language descriptions of the same work
- Skill-trust formula validates every claim: proficiency $\times$ time with skill $\times$ endorsements $\times$ platform assessment - a Marketing Manager listing “FAISS: Expert” scores near zero
- 23 behavioural platform signals evaluated - open-to-work, last active, recruiter response rate, notice period, GitHub activity, interview completion - quality and reachability

Ranking Methodology

Title Filter --> 31K --> Bi-Encoder --> Top 300 --> Cross-Encoder --> Top 100

- **Stage 1 - Title Pre-Filter (~2 s):** 17-category title relevance map eliminates 69,000 irrelevant profiles instantly. Business Analysts, HR Managers, Accountants → score 0.02. No model inference.
- **Stage 2 - Bi-Encoder + Composite Scoring (~15 s):** Pre-computed 384-dim embedding's compared against three JD query vectors via vectorized cosine similarity. Composite score: 35% skills + 30% career quality + 10% experience + 10% location + 5% education + platform quality. Top 300 shortlisted.
- **Stage 3 - Cross-Encoder Re-Ranking (~15–30 s):** Local ms-marco-MiniLM-L-6-v2 reads (JD query, candidate summary) pairs and assigns deep relevance scores. Final score = 40% composite + 60% cross-encoder. Top 100 selected.

Signal combination

- Behavioural availability multiplier applied post-scoring - smooth exponential decay across recency, response rate, open-to-work, notice period ($0.05\times$ to $1.30\times$)
- Six soft-penalty disqualifier curves applied multiplicatively - no binary cut-off's, proportional penalization

Explainability & Data Validation

Explainability

- Every top-100 candidate gets a 1-2 sentence reasoning grounded only in their real profile data - title, company, skills, years, location, notice period
- Six Stage-4 quality checks enforced: specific facts, JD connection, honest concerns, no hallucination, unique reasoning, tone consistent with rank
- Per-candidate score breakdown visible - each component's contribution, behavioural multiplier, and cross-encoder vs algorithmic score

Suspicious Profile Handling

- Evidence-accumulation honeypot detector across four checks: timeline consistency, expert proficiency vs actual duration, education-date logic, data coherence
- Soft confidence score (0-1), not binary - high-confidence honeypots set to 0.0 and excluded; borderline cases penalized proportionally
- Maintains below 10% contamination in top-100 - the hard disqualification threshold

Hallucination Prevention

- Grounding verification runs after reasoning - every claim cross-checked against its source field; unverified claims removed before output
- Template-based reasoning in competition mode ensures zero hallucination risk - only real data is inserted into fixed sentence structures

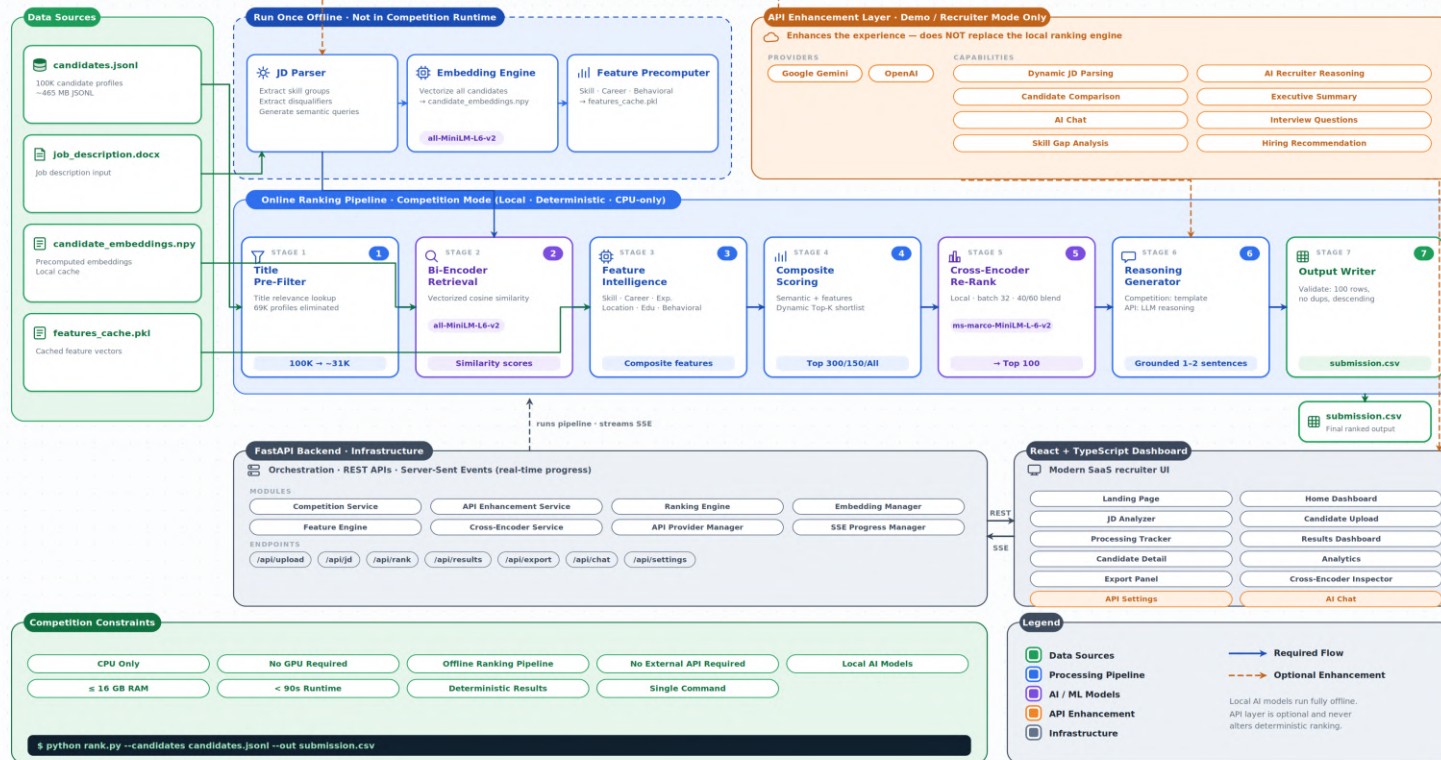
End-to-End Workflow

- Offline Pre-Computation (run once): Parse JD, extract 6 skill groups, 6 disqualifiers, 3 semantic queries. Embed 100K candidates with sentence-transformers, save candidate_embeddings.npy. Pre-compute all feature scores, save features_cache.pkl. Runs outside the 5-minute window.
- Online Ranking under 90 seconds: Load candidates.jsonl with null-safe field access. Title pre-filter eliminates 69K instantly. Load pre-computed embeddings, compute vectorized cosine similarity against 3 JD queries across 31K. Six-component scoring, sort, shortlist top 300.
- Re-Ranking and Output: Cross-encoder scores 300 pairs in batches of 32. Blend 40% composite + 60% CE. Select top 100. Generate grounded reasoning. Write submission.csv. Validate: 100 rows, non-increasing scores, below 10% honeypots.

System Architecture

RedroBRanker — AI Candidate Ranking System Architecture

Local-first deterministic ranking pipeline - optional API enhancement layer - single source of truth for ranking



Results & Performance

- Dataset reality: only 745 of 100,000 candidates hold genuine ML/AI titles (0.75% signal density). Core ML/AI profiles concentrated from 0.75% of full pool to approximately 65% of final top-100, an 87x concentration factor proving effective discrimination. Cross-encoder re-ranking shifted candidates an average of 8+ positions, promoting plain-language production engineers and demoting keyword-heavy non-engineers.
- Total online runtime under 90 seconds on CPU only: Title Filter 2s, Embedding similarity 5s, Feature scoring 10s, Cross-encoder re-ranking 30s, Output generation 3s. Fully compliant: CPU only, 16GB RAM, no GPU, no network during ranking. Single command: `python rank.py --candidates ./candidates.jsonl --out ./submission.csv`. All competition constraints met.

Technologies Used

Layer	Technology	Why Selected
Bi-Encoder	sentence-transformers all-MiniLM-L6-v2	384 dims, CPU-compatible, 80 MB, pre-computable offline - best speed/quality ratio
Cross-Encoder	HuggingFace ms-marco-MiniLM-L-6-v2	Trained on MS MARCO retrieval - directly applies to candidate-JD relevance. 117 MB, no network
Vector Operations	NumPy	Vectorized cosine similarity across 100K embeddings in seconds - no ML-framework overhead
Data Pipeline	Python + ijson + gzip	Streams 465 MB JSONL without loading into RAM. Auto-detects gzip vs plain
Backend API	FastAPI + Uvicorn + SSE	Async, lightweight, Server-Sent Events for real-time 8-stage pipeline progress
Frontend	React + Recharts + Lucide React	Component-based; Recharts for ranking analytics, Lucide for a consistent icon system
JD Parsing	python-docx + rule-based extraction	No LLM dependency during ranking - deterministic and reproducible
API Mode	Gemini 1.5 Flash + GPT-4o-mini	Near-zero cost for the sandbox demo - Flash for re-rank speed, GPT-4o-mini for JSON reliability

API Mode - Enhanced Capabilities

- API mode upgrades three stages: Google text-embedding-004 for embeddings, Gemini 1.5 Flash for LLM re-ranking of top-300, GPT-4o-mini or Claude Haiku for richer reasoning. Competition rank.py stays fully local - API mode is sandbox/demo only, meeting the no-network constraint.
- Six new capabilities: LLM re-ranking with structured per-candidate scores and reasons, dynamic JD parsing for any job role (not just Senior AI Engineer), conversational Q&A over ranked results, side-by-side candidate comparison, and AI executive brief generation for hiring managers.
- Cost is negligible: full 100K run on Google Gemini costs under \$0.02. Demo of 50 candidates under \$0.001. Four providers supported: OpenAI, Google Gemini, Anthropic Claude, Cohere. Automatic fallback to local models if any API call fails.

API Mode - New Screens & UI Changes

- Two new frontend screens: S0 API Settings (provider selection, API key input, per-task model selector, live cost estimator, automatic fallback toggle) and S12 AI Chat (conversational interface over ranked results, suggested questions, data-cited answers, session token counter).
- Five existing screens enhanced: S3 JD Analyzer adds dynamic JD input with Parse with AI button. S6 Processing shows LLM as Stage 8 with batch counter. S7 Results adds LLM Score column. S8 Candidate Detail adds Ask AI chat and Explain Score. S10 Export adds AI Executive Brief.
- Configuration toggle switches Competition Mode (local models, reproducible, matches rank.py submission) and Demo Mode (API-enhanced, richer reasoning). Both produce valid submission.csv. API keys stored in .env file - never hardcoded or committed to the repository.

Submission Assets

- GitHub URL : <https://github.com/Manish-111913/AI-JD.git>
- Frontend Production mode URL : <https://ai-jd-gamma.vercel.app>
- Backend Production URL : <https://huggingface.co/spaces/Manish111913/AI-JD>



INDIA.RUNS

Build what next India runs on

THANK YOU